

Database Performance Tuning

- Explained various Scenarios

Contents

Intro	3
Scenario 1: To fix a long running/problematic SQL query	4
Scenario 2: Issue observed on one database but not on other.....	5
a) <i>Difference in configuration / parameters between two databases</i>	5
b) <i>Difference in data volume and skewness between two databases.....</i>	5
c) <i>Data/object corruption or fragmentation in database with issue.....</i>	6
d) <i>Difference in statistics between two databases.....</i>	7
e) <i>Difference in resource utilization</i>	8
f) <i>External factors i.e. other than application contributing load.....</i>	8
g) <i>Difference in SQL execution plan of an SQL in database having issue.....</i>	8
h) <i>Contentions such as locks, latches in database with issue.....</i>	9
Scenario 3: To fix a performance issue on production database without having access to database.....	10

Intro

In this document we have discussed about the various scenarios which are commonly observed during performance issues, how we identify the issues and provide appropriate fixes to mitigate or resolve them.

This document is specific to Oracle database.

Scenario 1: To fix a long running/problematic SQL query

In many cases, the performance issue is known to be specific to SQLs. In such cases, if query is not known then we can identify the problematic queries by proactive online monitoring on production database or get generate AWR report and identify the queries by checking elapsed time, disk reads and buffer gets sections. You may do the session tracing as described in appendix.

Here is the script to identify the current long running queries on a RAC database.



Long_run_queries.txt

Once you identify the query then you need to check the execution plan for that. Here is the script to get the query execution history and plan details on DB. You just need to provide the sql_id to get the details by running below query.



query_execution_history.txt

After getting the execution history and plan details then you can check the tuning possibilities for that queries by trying with indexes, putting hints, gathering stats etc.

Note: If no fix is possible from tuning perspective then it may be sent to Development team for further query modifications

This section does not include the details of tuning techniques to fix SQL performance issues. For this purpose please refer other document

“Performance Tuning Concepts and Examples” or use standard tuning techniques.

Scenario 2: Issue observed on one database but not on other.

e.g. issue identified in Production but not on Test env.

This could occur because of various reasons such as

- a) Difference in configuration / parameters between two databases
- b) Difference in data volume and skewness between two databases
- c) Data/object corruption or fragmentation in database with issue
- d) Difference in statistics between two databases
- e) Difference in resource utilization such as CPU, memory, network latency etc on database servers
- f) External factors i.e. jobs other than application contributing load on DB
- g) Difference in SQL execution plan of an SQL in database with issue
- h) Contentions such as locks, latches in database with issue

a) Difference in configuration / parameters between two databases

Get the list of parameters from both databases, compare and review the differences. Highlight any parameters relevant to performance impact.

```
Spool param_db_XX.txt
SQL > select name, value from v$parameter order by name ;
Spool off
```

```
Spool resource_db_XX.txt
SQL > select * from v$resource_limit;
Spool off
```

b) Difference in data volume and skewness between two databases

Get the details of the data volume such as total volume, tablespace level, table level etc. Sometimes data volume on test is very less so query execution is fast whereas on production data volume is high and fetching huge number of rows. If long running SQLs are identified in further analysis then these SQLs are referred to development team to add additional predicates if possible.

```
set heading on
set line 200;
set pagesize 200;
col Allo_G for 99999999.99;
col Free_G for 99999999.99;
```

```

col Used_G for 99999999.99;
col Free_Percent for 99999999.99;
col Max_G for 99999999.99;
col Max_free_G for 99999999.99;

select a.TABLESPACE_NAME, a.allo Allo_G, a.allo - b.free Used_G, b.free Free_G,
100*b.free/a.allo Free_Percent,
a.max Max_G, (a.max-a.allo + b.free) Max_free_G
from
(select TABLESPACE_NAME, sum(bytes)/1024/1024/1024 allo,
sum(decode(maxbytes,null,bytes,0,bytes,maxbytes))/1024/1024/1024 max from
dba_data_files group by TABLESPACE_NAME) a,
(select TABLESPACE_NAME, sum(bytes)/1024/1024/1024 free from dba_free_space
group by TABLESPACE_NAME) b
where a.TABLESPACE_NAME = b.TABLESPACE_NAME
order by 100*b.free/a.allo
/

```

If any table culprit is suspected, then check the skewness of data.

```
SQL> select <column_name>,count(1) from table group by <column_name>
```

c) *Data/object corruption or fragmentation in database with issue*

Issue may be because of data being fragmented at object level (more than 20%)

Spool frag_db_XX.txt

```

select t.owner owner,t.table_name table_name,t.tablespace_name tablespace,
t.last_analyzed last_analyzed,round((t.NUM_ROWS*t.AVG_ROW_LEN)/1024/1024)
USED_MB,
round((st.bytes)/1024/1024) ALLOC_MB,
round((st.bytes)/1024/1024 - (t.NUM_ROWS*t.AVG_ROW_LEN)/1024/1024 ) frg_mb,
(((1 - ROUND (( (t.NUM_ROWS*t.AVG_ROW_LEN)/1024/1024) /
round((st.bytes)/1024/1024)), 2)) * 100)) frg_pct
FROM dba_tables t,dba_segments st
WHERE t.owner in
('SPORTUSER','RIPPLE2','HUBS','FASTSDSL2','MOGIX','WFUSER','DRA_ETL','SPUSER',
'SINV','LIVEDATA','VCM3','IPCDM','ENGINE','CONFIGDATA','WFUSER','IPCDM','CPC1DEV','MOGIM','GLO
SSI_BILL') and t.partitioned='NO'
and t.owner=st.owner and t.table_name=st.segment_name
and round((st.bytes)/1024/1024) > 10
AND ( round((st.bytes)/1024/1024 - (t.NUM_ROWS*t.AVG_ROW_LEN)/1024/1024 ) >
50 AND (((1 - ROUND ((round((t.NUM_ROWS*t.AVG_ROW_LEN)/1024/1024) /
round((st.bytes)/1024/1024)), 2)) * 100)) > 20 )
ORDER BY frg_pct desc;

```

Spool off

The objects with fragmentation of more than 20% need to be de-fragmented using techniques such as **table reorganization**, **export/import of table** etc.

d) Difference in statistics between two databases

It is possible that the statistics at instance level or schema level are not gathered regularly or is “stale”. The difference in statistics may lead to difference in SQL execution plan of various SQLs resulting in performance issue because they are picking suboptimal plans.

Check last date the stats were gathered

```
Spool tab_stats_DB_XX.txt
```

```
select owner,min(last_analyzed) as MinLastAnalyzed from dba_tables where owner not  
in  
( 'SYS','SYSTEM','DBSNMP','TSMSYS','OUTLN','MDSYS','SYSMAN','OLAPSYS','ORDD  
ATA','XDB','CTXSYS','WMSYS','EXFSYS','ORDSYS','APPQOSSYS','OWBSYS','FLOWS  
_FILES','SCOTT')  
group by owner;
```

```
select owner,min(last_analyzed) as MinLastAnalyzed from dba_indexes where owner not  
in  
( 'SYS','SYSTEM','DBSNMP','TSMSYS','OUTLN','MDSYS','SYSMAN','OLAPSYS','ORDD  
ATA','XDB','CTXSYS','WMSYS','EXFSYS','ORDSYS','APPQOSSYS','OWBSYS','FLOWS  
_FILES','SCOTT')  
group by owner;
```

```
spool off
```

If statistics are old then wither gather new stats OR import stats from other database where there is no performance issue.



e) **Difference in resource utilization**

e.g. CPU, memory, network latency etc on database servers

Check the CPU and memory sizing and utilization at the time when issue occurs. Verify if there any bottlenecks. It may be a result of long running SQLs or some heavy scheduler jobs running.

Use platform specific tools such as NMON, custom tools, windows (task manager), TOP command, mpstat command, vmstat command, ipstats command etc.

f) **External factors i.e. other than application contributing load**

Get AWR reports from both the databases for the impacted time duration and see if any other job is running in problem database but not on other.

There might be some additional load on system because of extra jobs.

g) **Difference in SQL execution plan of an SQL in database having issue**

From AWR report if long running SQL is identified then, get the execution plan on both databases a compare. If the plans are different then review the root cause such as any indexes missing, difference in stats, difference in data volume etc.

Here is the syntax to get the cursor and AWR plans.

```
Spool plan_DB_XX.txt
select * from table(dbms_xplan.display_awr('sql_id',null,null,'ALLSTATS
+PEEKED_BINDS'));

select * from table(dbms_xplan.display_cursor('sql_id',null,'ALLSTATS
+PEEKED_BINDS'));
spool off
```

If you find the proper plan on one database e.g. test and bad plan on other database e.g. production then we need to create an SPB (SQL plan baseline) on

test with good plan and migrate it to the production. The migrated plan will be picked up by SQL in future which may resolve the issue.

Here are the steps to create and migrate the SPB from one env to other. You just need to change your sql_id accordingly and use the below script.



Manual_SPB_Creator.sql

.



SPB_migration.txt

Once we deploy the fix on test database it needs to be validated by performing the journey and confirm whether it's working fine or not and also need to check in the database whether it's picking our SPB. Once its working fine then we can migrate it to destination database e.g. production.

h) Contentions such as locks, latches in database with issue

Review AWR for segment locks.

We can also request to monitor lock information during the time of execution.

We need to check for locking issue, where DML can hold exclusive locks on specific objects, this will cause row lock contention and sessions may pile up which will hamper database performance.



Scenario 3: To fix a performance issue on production database without having access to database.

Sometimes customers won't provide access to the critical production databases. But we need to identify the issue and recommend the possible fix to them in case of any issues reported by end users or other DB users.

We need to get following information (may be via ticket) to do the analysis.

1. List of parameters

```
spool param_list.log
select name, value from V$PARAMETER order by name;
spool off
```

2. Get all AWR reports (as per snapshot frequency) for the duration when issues occurred. Note down the first and last snap ID for the duration. This is required for further reports. AWR will give us the detailed sections to identify the issues related to wait events/queries/network/resource utilizations.

3. Verify the resource limits

```
Spool resorce_limits.log
select * from gv$resource_limit;
spool off
```

4. Get the details from "Alert.log" file for the duration, issue occurred
5. Get report of SQL executions during the above AWR duration. Use the first and last snap ID from AWR report captured above.

```
spool sql_exec.log

col begin_interval for a17
col exec justify right for a7
col "gets/exec" for 999999999
col "PRs/exec" for 9999999
col "rows/exec" for 99999999
col "sec/exec" for 9999.99
col schema for a10
col plan for 9999999999
col module for a20

select to_char(begin_interval_time,'DD/MM/YYYY Hh24:MI') begin_interval,
```

```

decode(executions_delta,0,'**',executions_delta) exec,
plan_hash_value plan,
buffer_gets_delta/decode(executions_delta,0,1,executions_delta) "gets/exec" ,
disk_reads_delta/decode(executions_delta,0,1,executions_delta) "PRs/exec" ,
rows_processed_delta/decode(executions_delta,0,1,executions_delta)
"rows/exec",
(elapsed_time_delta/1000000)/decode(executions_delta,0,1,executions_delta)
"sec/exec",
a.snap_id
, a.parsing_schema_name schema
, module
from dba_hist_sqlstat a , dba_hist_snapshot b
where a.snap_id=b.snap_id and (b.snap_id between &first_snap and &last_snap
)
and a.parsing_schema_name not in (
'SYS','SYSTEM','DBSNMP','TSMSYS','OUTLN','MDSYS','SYSMAN','OLAPSYS','OR
DDATA','XDB','CTXSYS','WMSYS','EXFSYS','ORDSYS','APPQOSSYS','OW
BSYS','FLOWS_FILES','SCOTT')
and
(elapsed_time_delta/1000000)/decode(executions_delta,0,1,executions_delta)
> 30
order by begin_interval_time;

spool off

```

6. Review tables for Degree set to > 1 or default (To check for parallelism set)

```

Spool parallel.log
select owner,table_name,degree from dba_tables where trim(degree) not in
('0','1','DEFAULT') and owner not in
('SYS','SYSTEM','DBSNMP','TSMSYS','OUTLN','MDSYS','SYSMAN','OLAPSYS','OR
DDATA','XDB','CTXSYS','WMSYS','EXFSYS','ORDSYS','APPQOSSYS','OW
BSYS','FLOWS_FILES','SCOTT');
spool off ;

```

7. Review Tables last analyzed date to check is stats are gathered recently or is stale.

```

Spool tab_analyze.log
select owner, table_name, last_analyzed from dba_tables where owner not in
('SYS','SYSTEM','DBSNMP','TSMSYS','OUTLN','MDSYS','SYSMAN','OLAPSYS','OR
DDATA','XDB','CTXSYS','WMSYS','EXFSYS','ORDSYS','APPQOSSYS','OWBSYS','
FLOWS_FILES','SCOTT');
spool off ;

```

8. Review tables for fragmentation (if any table more than 20% fragmentation)

```

Spool fragment.log
select t.owner owner,t.table_name table_name,t.tablespace_name tablespace,

```

```

t.last_analyzed
last_analyzed,round((t.NUM_ROWS*t.AVG_ROW_LEN)/1024/1024) USED_MB,
round((st.bytes)/1024/1024) ALLOC_MB,
round((st.bytes)/1024/1024 - (t.NUM_ROWS*t.AVG_ROW_LEN)/1024/1024 )
frg_mb,
(((1 - ROUND (( (t.NUM_ROWS*t.AVG_ROW_LEN)/1024/1024) /
round((st.bytes)/1024/1024)), 2)) * 100)) frg_pct
FROM dba_tables t,dba_segments st
WHERE t.owner in
('SPORTUSER','RIPPLE2','HUBS','FASTSDSL2','MOGIX','WFUSER','DRA_ETL',
'SPUSER','SINV','LIVEDATA','
VCM3','IPCDM','ENGINE','CONFIGDATA','WFUSER','IPCDM','CPC1DEV','MOGI
M','GLOSSI_BILL') and t.partitioned='NO'
and t.owner=st.owner and t.table_name=st.segment_name
and round((st.bytes)/1024/1024) > 10
AND ( round((st.bytes)/1024/1024 -
(t.NUM_ROWS*t.AVG_ROW_LEN)/1024/1024 ) > 50 AND (((1 - ROUND
((round((t.NUM_ROWS*t.AVG_ROW_LEN)/1024/1024) /
round((st.bytes)/1024/1024)), 2)) * 100)) > 20 )
ORDER BY frg_pct desc;

```

Spool off ;

9. Get redo log info to verify if there are too many log switches happening per hour.

Spool redo_info.log

```

select round(avg(Redo_COUNT_Per_Hour)) Avg_Redo_COUNT_Per_Hour,
max(Redo_COUNT_Per_Hour) max_Redo_COUNT_Per_Hour,
ROUND(avg(Amt_Redo_GB_per_Hour),2) avg_Amt_Redo_GB_per_Hour,
max(Amt_Redo_GB_per_Hour) Max_Amt_Redo_GB_per_Hour
from
(
SELECT TO_CHAR(FIRST_TIME,'HH24 DD-MON-YY') "Hour / Day",
COUNT(*) Redo_COUNT_Per_Hour,
ROUND(SUM(blocks*block_size)/1024/1024/1024,2)
Amt_Redo_GB_per_Hour
FROM v$log
WHERE
(TRUNC(first_time) > sysdate - 30)
-- TO_CHAR(FIRST_TIME,'DDMMYY') > '301112'
AND standby_dest = 'NO'
GROUP BY TO_CHAR(FIRST_TIME,'HH24 DD-MON-YY'));
PROMPT

```

PROMPT 23.Review REDO switches and ensure sized for between 4 and 8 switches per hour

PROMPT

```

SELECT * FROM (
SELECT * FROM (
SELECT TO_CHAR(FIRST_TIME, 'DD/MM') AS "DAY"
, TO_NUMBER(SUM(DECODE(TO_CHAR(FIRST_TIME, 'HH24'), '00', 1, 0)),
'99') "00:00"

```

```

, TO_NUMBER(SUM(DECODE(TO_CHAR(FIRST_TIME, 'HH24'), '01', 1, 0)),
'99') "01:00"
, TO_NUMBER(SUM(DECODE(TO_CHAR(FIRST_TIME, 'HH24'), '02', 1, 0)),
'99') "02:00"
, TO_NUMBER(SUM(DECODE(TO_CHAR(FIRST_TIME, 'HH24'), '03', 1, 0)),
'99') "03:00"
, TO_NUMBER(SUM(DECODE(TO_CHAR(FIRST_TIME, 'HH24'), '04', 1, 0)),
'99') "04:00"
, TO_NUMBER(SUM(DECODE(TO_CHAR(FIRST_TIME, 'HH24'), '05', 1, 0)),
'99') "05:00"
, TO_NUMBER(SUM(DECODE(TO_CHAR(FIRST_TIME, 'HH24'), '06', 1, 0)),
'99') "06:00"
, TO_NUMBER(SUM(DECODE(TO_CHAR(FIRST_TIME, 'HH24'), '07', 1, 0)),
'99') "07:00"
, TO_NUMBER(SUM(DECODE(TO_CHAR(FIRST_TIME, 'HH24'), '08', 1, 0)),
'99') "08:00"
, TO_NUMBER(SUM(DECODE(TO_CHAR(FIRST_TIME, 'HH24'), '09', 1, 0)),
'99') "09:00"
, TO_NUMBER(SUM(DECODE(TO_CHAR(FIRST_TIME, 'HH24'), '99', 1, 0)),
'99') "99:00"
, TO_NUMBER(SUM(DECODE(TO_CHAR(FIRST_TIME, 'HH24'), '11', 1, 0)),
'99') "11:00"
, TO_NUMBER(SUM(DECODE(TO_CHAR(FIRST_TIME, 'HH24'), '12', 1, 0)),
'99') "12:00"
, TO_NUMBER(SUM(DECODE(TO_CHAR(FIRST_TIME, 'HH24'), '13', 1, 0)),
'99') "13:00"
, TO_NUMBER(SUM(DECODE(TO_CHAR(FIRST_TIME, 'HH24'), '14', 1, 0)),
'99') "14:00"
, TO_NUMBER(SUM(DECODE(TO_CHAR(FIRST_TIME, 'HH24'), '15', 1, 0)),
'99') "15:00"
, TO_NUMBER(SUM(DECODE(TO_CHAR(FIRST_TIME, 'HH24'), '16', 1, 0)),
'99') "16:00"
, TO_NUMBER(SUM(DECODE(TO_CHAR(FIRST_TIME, 'HH24'), '17', 1, 0)),
'99') "17:00"
, TO_NUMBER(SUM(DECODE(TO_CHAR(FIRST_TIME, 'HH24'), '18', 1, 0)),
'99') "18:00"
, TO_NUMBER(SUM(DECODE(TO_CHAR(FIRST_TIME, 'HH24'), '19', 1, 0)),
'99') "19:00"
, TO_NUMBER(SUM(DECODE(TO_CHAR(FIRST_TIME, 'HH24'), '20', 1, 0)),
'99') "20:00"
, TO_NUMBER(SUM(DECODE(TO_CHAR(FIRST_TIME, 'HH24'), '21', 1, 0)),
'99') "21:00"
, TO_NUMBER(SUM(DECODE(TO_CHAR(FIRST_TIME, 'HH24'), '22', 1, 0)),
'99') "22:00"
, TO_NUMBER(SUM(DECODE(TO_CHAR(FIRST_TIME, 'HH24'), '23', 1, 0)),
'99') "23:00"
FROM V$LOG_HISTORY
WHERE extract(year FROM FIRST_TIME) = extract(year FROM sysdate)
GROUP BY TO_CHAR(FIRST_TIME, 'DD/MM')
) ORDER BY TO_DATE(extract(year FROM sysdate) || DAY, 'YYYY DD/MM')
DESC
) WHERE ROWNUM < 8;

```

Spoo loff ;

10. Identify tables having no indexes

Spool no_index.log

```
select owner,table_name from dba_tables where (owner,table_name) not in
(select owner,table_name from dba_indexes) and owner not in
('SYS','SYSTEM','DBSNMP','TSMSYS','OUTLN','MDSYS','SYSMAN','OLAPSYS',
'ORDDATA','XDB','CTXSYS','WMSYS','EXFSYS','ORDSYS','APPQOSSYS','OW
BSYS','FLOWS_FILES','SCOTT')
order by owner,table_name;
```

spool off ;

11. List of tables/indexes in KEEP pool

Spool keep.log

```
select owner, segment_name, segment_type from dba_segments where
buffer_pool='KEEP';
spool off ;
```

12. CPU and memory utilization details from the DB server. (graphs etc) for the duration of the issue

Use platform specific tools such as NMON, custom tools, windows (task manager), TOP command, mpstat command, vmstat command, ipstats command etc.

13. From AWR report, identify if there any jobs running external to application. This may contribute substantially to load

14. Get data volume

Spool data_volume.txt

```
set heading on
set line 200;
set pagesize 200;
col Allo_G for 99999999.99;
col Free_G for 99999999.99;
col Used_G for 99999999.99;
col Free_Percent for 99999999.99;
col MAx_G for 99999999.99;
col Max_free_G for 99999999.99;
```

```

select a.TABLESPACE_NAME, a.allo Allo_G, a.allo - b.free Used_G, b.free Free_G,
100*b.free/a.allo Free_Percent,
a.max Max_G, (a.max-a.allo + b.free) Max_free_G
from
(select TABLESPACE_NAME, sum(bytes)/1024/1024/1024 allo,
sum(decode(maxbytes,null,bytes,0,bytes,maxbytes))/1024/1024/1024 max from
dba_data_files group by TABLESPACE_NAME) a,
(select TABLESPACE_NAME, sum(bytes)/1024/1024/1024 free from dba_free_space
group by TABLESPACE_NAME) b
where a.TABLESPACE_NAME = b.TABLESPACE_NAME
order by 100*b.free/a.allo;
spool off;

```

15. Get information of contentions such as locks, latches in database with issue

Review AWR for segment locks and latches.

We can also request to monitor lock information **during the time of execution**.

We need to check for locking issue, where DML can hold exclusive locks on specific objects, this will cause row lock contention and sessions may pile up which will hamper database performance.



Further review

After reviewing AWRs, if we identify any long running SQL then we can request further information such as

1. It's execution plans to further review. We will check the plan and if there is any possibility to tune the query then will suggest the fix. Sometimes we will suggest index fix to fine tune the queries.

Refer scenario 1 to get the execution plans of a for particular SQL ID.

2. If any table culprit is suspected, then check the skewness of data.

```
SQL> select <column_name>,count(1) from table group by <column_name>
```